

## Client:Shopify Inc

### Current Challenges:

- ❖ Data scattered across MySQL, Salesforce, SAP HANA, POS, and Teradata, creating data silos.
- ❖ Manual ETL processes increase processing time and operational effort.
- ❖ Delayed reporting affects timely business decisions.
- ❖ Existing platform struggles to scale with growing data volumes.
- ❖ Duplicate and inconsistent data across multiple systems.
- ❖ Limited monitoring and orchestration of ETL workflows.
- ❖ Weak data governance and metadata management.
- ❖ High infrastructure maintenance and operational costs.

### Proposed High-Level Solution:

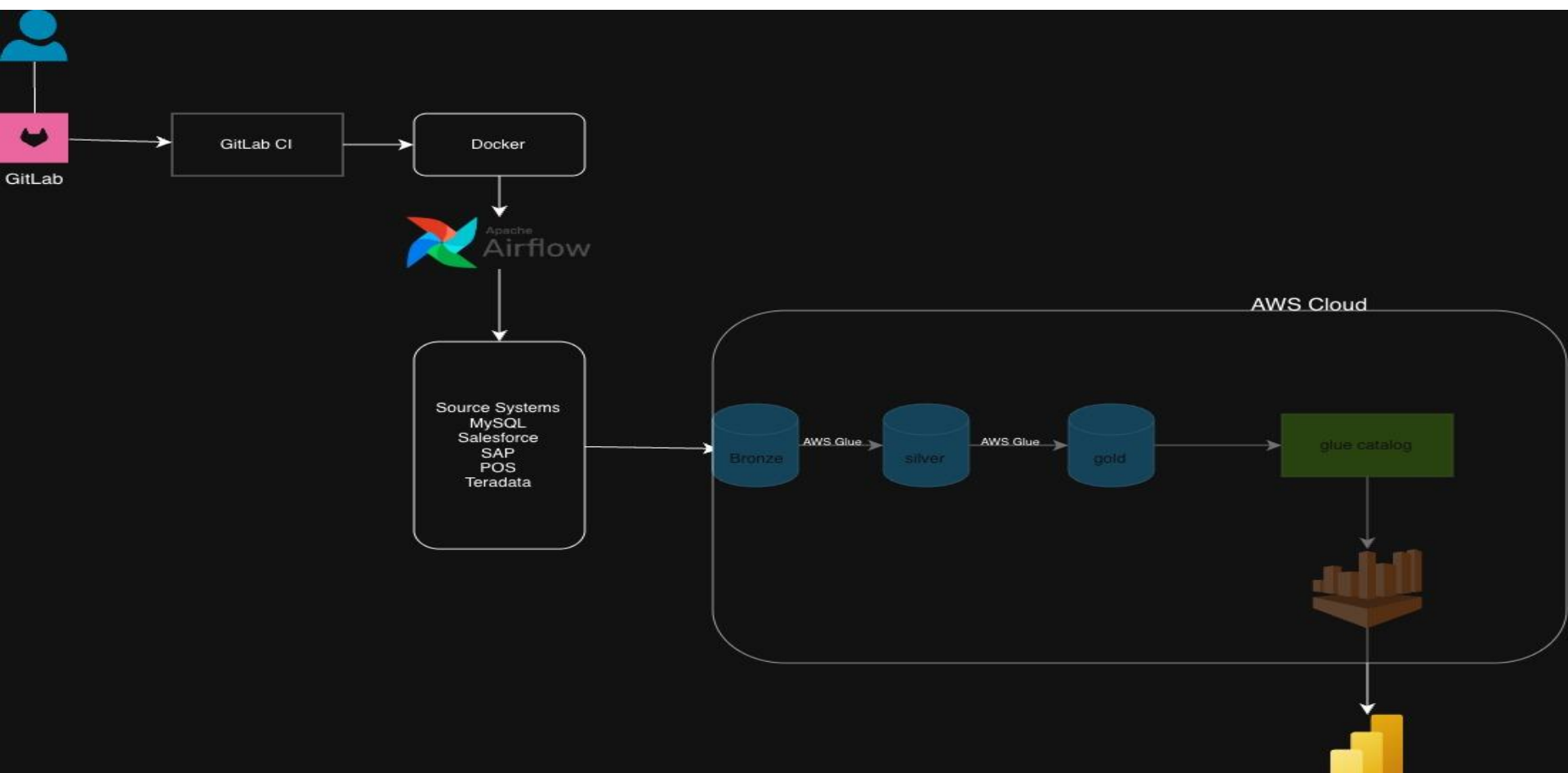
AWS Medallion Architecture (Bronze → Silver → Gold)

Technology Stack:Amazon S3 – Data Lake - AWS Glue – ETL & Data Transformation - Apache Airflow – Workflow Orchestration - AWS Glue Data Catalog – Metadata Management - Amazon Athena – SQL Analytics - GitLab & GitLab CI/CD – Version Control & CI/CD

### Expected Benefits:

- Automated ETL Pipelines
- Faster Analytics & Reporting
- Improved Data Quality & Governance
- Scalable Cloud Architecture
- Reduced Operational Cost

# Shopify Medallion Architecture on AWS:



# DAILY & WEEKLY JOB SCHEDULING

## Daily Scheduled Jobs:

| Job Name                      | Description                             |
|-------------------------------|-----------------------------------------|
| Customer_ETL                  | →Ingest customer data from MySQL        |
| Product_ETL                   | →Load product master data from SAP HANA |
| Orders_ETL                    | →Process POS sales orders               |
| Payments_ETL                  | →Process payment data from Salesforce   |
| Inventory_ETL                 | →Load inventory data from Teradata      |
| Gold_Dim_Customer             | →Build Customer Dimension               |
| Gold_Dim_Product              | →Build Product Dimension                |
| Gold_Fact_Orders              | →Create Orders                          |
| Fact Table Gold_Fact_Payments | →Create Payments Fact Table             |
| Gold_Inventory_Summary        | →Generate Inventory Summary             |

## Weekly Scheduled Jobs:

| Job Name                 | Purpose                            |
|--------------------------|------------------------------------|
| Weekly_Sales_Summary     | →Generate Weekly Sales Report      |
| Inventory_Reconciliation | →Validate Inventory Records        |
| Data_Archive             | →Archive Historical Data           |
| Data_Cleanup             | →Remove Temporary & Obsolete Files |
| Glue_Catalog_Refresh     | →Refresh AWS Glue Data Catalog     |

Schedule Daily

Schedule: 02:00 AM IST (Apache Airflow)

Weekly Schedule: Every Sunday – 03:00 AM IST

# DEV, TEST & PROD ENVIRONMENTS WITH GITLAB CI/CD :

## Development (DEV):

Developers build new ETL pipelines and Airflow DAGs.

- Code is developed in feature branches.
- Unit testing is performed before committing code.

## Testing (TEST):

Code is merged into the develop branch.

- GitLab CI automatically validates the code.
- Integration and functional testing are performed.
- Pipeline execution and data validation are verified.

## Production (PROD):

After successful testing, the code is merged into the main branch.

- CAB (Change Advisory Board) approval is obtained.
- Production deployment is triggered through GitLab CI/CD.
- Airflow schedules and executes the production workflows.

## GitLab CI/CD Deployment Process

1. Developer creates a Feature Branch.
  2. Develop ETL code and Airflow DAGs.
  3. Commit and Push changes to GitLab.
  4. Raise a Merge Request (MR).
  5. GitLab CI runs automated validation.
  6. Code Review and Approval.
  7. Merge into Develop Branch.
  8. CAB Approval for Production Release.
  9. Merge into Main Branch.
  10. Deploy to Production and Monitor Airflow DAGs.
- Benefits • Automated deployments • Faster release cycle • Reduced manual effort • Improved code quality • Reliable and controlled production deployments

## Benefits:

- Automated deployments
- Faster release cycle
- Reduced manual effort
- Improved code quality
- Reliable and controlled production deployments

## 02-aws-shopify-pipeline/

|                                                                                   |                           |                                                 |
|-----------------------------------------------------------------------------------|---------------------------|-------------------------------------------------|
|  | <b>airflow/</b>           | Apache Airflow DAGs and configurations          |
|  | <b>config/</b>            | Configuration files and environment variables   |
|  | <b>docs/</b>              | Project documentation and architecture diagrams |
|  | <b>glue_jobs/</b>         | AWS Glue ETL job scripts                        |
|  | <b>queries/</b>           | Athena/SQL queries for analytics and validation |
|  | <b>scripts/</b>           | Utility scripts for setup and maintenance       |
|  | <b>tests/</b>             | Test scripts and validation checks              |
|  | <b>utils/</b>             | Reusable utility functions and helpers          |
|  | <b>views/</b>             | Saved views and analysis queries                |
|  | <b>Dockerfile</b>         | Docker image definition for Airflow             |
|  | <b>docker-compose.yml</b> | Docker Compose configuration for Airflow        |
|  | <b>requirements.txt</b>   | Python dependencies for the project             |
|  | <b>.gitlab-ci.yml</b>     | GitLab CI pipeline configuration                |
|  | <b>README.md</b>          | Project overview and setup instructions         |

## INITIAL ANALYSIS & DATA DISCOVERY PHASE:

### Initial Analysis Phase

- Gather business requirements and understand project objectives.
- Identify source systems (MySQL, Salesforce, SAP HANA, POS, Teradata).
- Understand data flow from source to target.
- Define project scope, timeline, and deliverables.
- Identify security, compliance, and governance requirements.
- Select appropriate AWS services and architecture.
- Estimate data volume, frequency, and storage requirements.
- Prepare high-level solution design.

### Data Discovery Phase

- Analyze source tables, columns, and relationships.
- Profile data to identify nulls, duplicates, and inconsistencies.
- Understand business rules and data transformations.
- Define primary keys and foreign key relationships.
- Identify incremental load columns (Created\_Date, Updated\_Date).
- Map source fields to target Bronze, Silver, and Gold layers.
- Identify data quality issues and validation rules.
- Document metadata, schemas, and table structures.

### Deliverables:

- Source-to-Target Mapping (STM)
- Data Dictionary
- High-Level Architecture
- ETL Design Document
- Data Quality Rules
- Pipeline Development Plan

## INCREMENTAL DATA HANDLING, LATE ARRIVING DATA & SCD TYPE 2:

### Incremental Data Handling:

- Incremental loading is implemented to process only newly added or modified records.
- Created\_Date and Updated\_Date columns are used to identify new records.
- This approach reduces ETL execution time and processing cost.
- Data is loaded into Bronze, transformed in Silver, and published to Gold.

### Late Arriving Data (Proposed Approach):

- Identify records that arrive after the scheduled ETL execution.
- Capture late records in the next pipeline run.
- Update the corresponding Silver and Gold tables.
- Maintain data consistency without duplicate records.

### SCD Type 2 (Future Enhancement):

- Track historical changes in dimension tables.
- Maintain Current\_Flag, Effective\_Date, and End\_Date columns.
- Insert a new record whenever a business attribute changes.
- Preserve historical versions for auditing and reporting.

### Schema Drift Handling (Proposed Approach):

- Detect new or modified columns automatically.
- Use AWS Glue DynamicFrames for schema evolution.
- Validate schema before loading data.
- Log schema changes and notify the support team.

### Benefits:

- Faster incremental processing
- Historical data tracking
- Improved data consistency
- Better schema management
- Scalable ETL pipeline

## DATA MODELLING IN THE GOLD LAYER:

### Gold Layer Purpose

- The Gold layer stores business-ready, curated, and aggregated data.
- It is optimized for reporting, dashboards, and analytical queries.
- Data is organized using a Star Schema for improved query performance.

### Gold Schema

Schema Name: gold

### Dimension Tables

- dim\_customer
  - customer\_id
  - customer\_name
  - city
  - state
  - country
- dim\_product
  - product\_id
  - product\_name
  - category
  - brand

### Fact Tables

- fact\_orders
  - order\_id
  - customer\_id
  - product\_id
  - order\_date
  - quantity
  - sales\_amount

- fact\_payments
  - payment\_id
  - order\_id
  - payment\_method
  - payment\_status
  - payment\_amount

### Summary Table

- inventory\_summary
  - product\_id
  - available\_quantity
  - warehouse\_location
  - last\_updated

### Benefits

- Optimized for business reporting
- Faster analytical queries
- Reduced data redundancy
- Improved dashboard performance
- Easy integration with Amazon Athena and BI tools



## DATA QUALITY CHECKS AT THE SILVER LAYER:

### Purpose:

- Ensure data accuracy, completeness, consistency, and reliability before publishing to the Gold layer.

### Data Quality Checks Performed

#### 1. Null Value Validation:

- Verify that mandatory columns do not contain NULL values.
- Example: customer\_id, order\_id, product\_id.

#### 2. Duplicate Record Validation:

- Identify and remove duplicate records using primary keys.
- Ensure only unique records are loaded into the Silver layer.

#### 3. Source-to-Target Record Count Validation:

- Compare the number of records between source and target.
- Ensure all valid records are successfully loaded.

#### 4. Data Type & Format Validation

- Validate data types such as Integer, Date, Timestamp, and Decimal.
- Reject or log invalid records for further analysis.

### Source → Target Validation Process

Source System



Extract Data



Data Quality Checks



Silver Layer



Validation Report



Gold Layer

### Benefits:

- Improved data accuracy
- Reliable business reporting
- Reduced data inconsistencies
- Higher data quality for analytics
- Better trust in business decisions

# SCHEMA NAMES, TABLES & STORED PROCEDURES:

## Schema Names

- bronze
- silver
- gold

## Bronze Layer Tables

- bronze\_customer
- bronze\_product
- bronze\_orders
- bronze\_payments
- bronze\_inventory

## Silver Layer Tables

- silver\_customer
- silver\_product
- silver\_orders
- silver\_payments
- silver\_inventory

## Gold Layer Tables

- dim\_customer
- dim\_product
- fact\_orders
- fact\_payments
- inventory\_summary

## Sample Stored Procedures (Future Enhancement)

- sp\_load\_customer\_data
- sp\_load\_product\_data
- sp\_generate\_sales\_summary
- sp\_refresh\_inventory
- sp\_data\_quality\_validation

## Purpose

- Bronze Layer – Stores raw source data.
- Silver Layer – Stores cleansed and transformed data.
- Gold Layer – Stores business-ready analytical datasets.
- Stored Procedures – Can be used in future warehouse implementations for automation and reporting.

DISASTER RECOVERY, COST OPTIMIZATION & PROJECT ESTIMATION:

Disaster Recovery Strategy

- Enable Amazon S3 Versioning to recover deleted or modified files.
- Store data across multiple Availability Zones (Multi-AZ) for high availability.
- Configure automated backups for critical datasets.
- Monitor ETL jobs using Apache Airflow and AWS CloudWatch.
- Implement IAM policies to secure AWS resources.
- Restore failed jobs using Airflow retry mechanisms.

Cost Optimization Techniques

- Store data in Amazon S3 instead of traditional data warehouses.
- Use partitioning to reduce Athena query costs.
- Optimize AWS Glue jobs to minimize execution time.
- Schedule ETL pipelines only when required.
- Archive historical data using Amazon S3 Lifecycle Policies.
- Monitor resource utilization and remove unused AWS resources.

Estimated Monthly Project Cost (Sample)

- Amazon S3: USD 10
- AWS Glue: USD 20
- Amazon Athena: USD 10
- AWS Glue Data Catalog: USD 5
- Apache Airflow (EC2/Docker): USD 15
- CloudWatch & Monitoring: USD 5

Estimated Total Cost: USD 65 / Month

Business Benefits

- High Availability
- Improved Reliability
- Lower Operational Cost
- Faster Disaster Recovery
- Scalable Cloud Infrastructure

# ORCHESTRATION USING APACHE AIRFLOW:

## Purpose:

- Automates and schedules the complete ETL workflow.
- Executes AWS Glue jobs in the correct sequence.
- Monitors pipeline execution and handles failures.

## Workflow

Start

↓

Customer\_ETL

↓

Product\_ETL

↓

Orders\_ETL

↓

Payments\_ETL

↓

Inventory\_ETL

↓

Gold Layer Jobs

↓

End

## Scheduling

- Daily ETL : 02:00 AM IST
- Weekly Jobs : Sunday – 03:00 AM IST
- Automatic Retry & Email Alerts

## Airflow Operators

- EmptyOperator
- GlueJobOperator
- PythonOperator

## Benefits

- Automated Scheduling
- Dependency Management
- Failure Recovery
- Real-Time Monitoring
- Scalable ETL Orchestration

# UPSTREAM & DOWNSTREAM APPLICATIONS:

## Upstream Systems

- MySQL
- Salesforce
- SAP HANA
- POS
- Teradata

## Downstream Teams

- Sales Team
- Finance Team
- Inventory Team
- Operations Team
- Marketing Team
- Business Analysts
- Management

## Dashboards & Reports

- Sales Performance Dashboard
- Customer Analytics Dashboard
- Product Performance Dashboard
- Inventory Monitoring Dashboard
- Revenue & Payment Dashboard
- Executive KPI Dashboard
- Operational Performance Report

## Business Benefits

- Real-time Reporting
- Better Decision Making
- Improved Customer Insights
- Inventory Optimization
- Business Performance Monitoring

## A DAY IN THE LIFE OF A DATA ENGINEER:

- Check Airflow DAG status and ETL job health.
- Monitor AWS Glue job executions.
- Validate Bronze, Silver & Gold layer data.
- Investigate and resolve pipeline failures.
- Optimize SQL queries and Glue ETL jobs.
- Review GitLab merge requests and CI pipeline.
- Support business requests and data validation.
- Update project documentation and attend Scrum meetings.

## OVERALL LEARNING & EXPERIENCE:

- Gained hands-on experience with AWS Data Engineering.
- Learned Medallion Architecture implementation.
- Improved ETL development and orchestration skills.
- Understood GitLab CI/CD workflow.
- Enhanced debugging and problem-solving abilities.
- Built confidence in designing scalable data pipelines.
- This project strengthened my practical Data Engineering knowledge.